PYTHON re

A Practical Guide to Regular Expressions

A clear, exam-friendly reference for pattern matching, text extraction, cleaning, and NLP assignments.

Learn	Use it for
Pattern syntax	Describing the text you want to find
re functions	Searching, extracting, replacing, splitting, validating
NLP examples	Cleaning text, extracting numbers, emails and URLs
Practice	Worked examples and quick revision questions

Core idea: A regular expression describes a *pattern*. The `re` function decides what Python does with the matching text.

Contents

1. What is re?
2. Regex syntax: character classes, quantifiers, boundaries, groups
3. The main re functions
4. Raw strings and flags
5. Practical NLP patterns
6. Common mistakes
7. Quick reference sheet
8. Practice questions with answers

1. What is re?

`re` is Python's built-in regular-expression module. It lets you find, extract, replace, and split text using patterns. Example: extract a ticket number and an email address from a sentence.

```
import re

text = "Ticket 12345: contact help@example.com"
numbers = re.findall(r"\d+", text)
emails = re.findall(r"[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}", text)
print(numbers)   # ['12345']
print(emails)    # ['help@example.com']
```

2. Regex syntax

2.1 Character classes

Pattern	Meaning	Example match
`\d`	One digit	7
`\D`	One non-digit	a
`\w`	Word character (letter, digit, underscore)	h, 2, _
`\W`	Non-word character	@
`\s`	Whitespace (space, tab, newline)	space
`\S`	Non-whitespace	A
`.`	Any character except newline (by default)	!
`[abc]`	One of a, b, c	b
`[a-z]`	Lowercase ASCII letter	m
`[A-Z]`	Uppercase ASCII letter	M
`[^0-9]`	Any character except a digit	x

2.2 Quantifiers

Pattern	Meaning	Example
`*`	Zero or more	`ab*c` matches ac, abc, abbc
`+`	One or more	`ab+c` matches abc, abbc (not ac)
`?`	Zero or one (optional)	`colou?r` matches color, colour
`{n}`	Exactly n repetitions	`\d{4}` matches 2026
`{n,}`	At least n repetitions	`\d{2,}` matches 42, 2026, 12345
`{n,m}`	Between n and m repetitions	`\d{2,4}` matches 42, 2026

2.3 Anchors and boundaries

Pattern	Meaning	Example
^	Start of string (start of each line with MULTILINE)	^Python
$	End of string (end of each line with MULTILINE)	end$
\b	Word boundary	\bcat\b matches cat, not category
\B	Not a word boundary	\Bcat matches the cat inside scatter, not at the start of cat or category

2.4 Groups and alternatives

Pattern	Meaning
(abc)	Capturing group; its captured text is what findall() returns.
(?:abc)	Non-capturing group; groups without capturing.
cat|dog	Alternation: match cat or dog.
(ab)+	One or more repetitions of ab. Note: with a capturing group, findall() returns only the last repetition ('ab'); use (?:ab)+ to get the whole match.

3. The main re functions

3.1 search(), match(), and fullmatch()

`search()` finds the first match anywhere. `match()` checks only from the beginning. `fullmatch()` requires the entire string to match.

```
import re
text = "Python is powerful."

print(bool(re.search(r"powerful", text)))     # True
print(bool(re.match(r"powerful", text)))      # False (text does not start with it)
print(bool(re.fullmatch(r"\w+", "Python")))   # True
print(bool(re.fullmatch(r"\w+", "Python 3"))) # False (the space breaks it)
```

3.2 findall()

Returns all non-overlapping matches as a list. With capturing groups, the list contains the captured groups (tuples if there are several groups).

```
import re
text = "Tickets 101 and 205 are open."
print(re.findall(r"\d+", text))
# ['101', '205']

print(re.findall(r"([A-Z])(\d+)", "A12 B34"))
# [('A', '12'), ('B', '34')]
```

3.3 sub()

Replaces each non-overlapping match with replacement text.

```
import re
text = "Email help@example.com now!"
cleaned = re.sub(r"[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}", "<EMAIL>", text)
print(cleaned)
# Email <EMAIL> now!
```

3.4 split()

Splits a string wherever the pattern matches.

```
import re
text = "apple,banana;orange mango"
print(re.split(r"[,;\s]+", text))
# ['apple', 'banana', 'orange', 'mango']
```

3.5 compile() and escape()

```
import re
pattern = re.compile(r"\b[A-Z][a-z]+\b")
print(pattern.findall("Python and Java are useful."))
# ['Python', 'Java']

literal = re.escape("price $10.50")
print(re.search(literal, "The price $10.50 is listed.").group())
# price $10.50
```

`compile()` creates a reusable pattern object. `escape()` makes special characters in literal text safe to use inside a pattern (here, $ and `.` are treated as plain characters).

4. Raw strings and flags

Prefer raw string literals such as `r"\d+"`. They keep backslashes intact for the regex engine and avoid Python escape-sequence problems.

```
import re
text = "Python python PYTHON"
print(re.findall(r"python", text, flags=re.IGNORECASE))
# ['Python', 'python', 'PYTHON']
```

Flag	Effect
`re.IGNORECASE / re.I`	Case-insensitive matching
`re.MULTILINE / re.M`	^ and $ match at each line boundary
`re.DOTALL / re.S`	. also matches newline characters
`re.VERBOSE / re.X`	Whitespace and comments allowed, so patterns are easier to read

5. Practical NLP patterns

5.1 Remove punctuation

```
import re
text = "Hello, World! NLP is amazing."
cleaned = re.sub(r"[^\w\s]", "", text)
print(cleaned)
# Hello World NLP is amazing
```

Inside a character class, `^` at the start means NOT. So `[^\w\s]` matches any character that is neither a word character nor whitespace.

5.2 Extract words with six or more letters

```
import re
text = "Python makes language processing interesting."
print(re.findall(r"\b[A-Za-z]{6,}\b", text))
# ['Python', 'language', 'processing', 'interesting']
```

5.3 Extract integers, decimals, and version numbers

```
import re
text = "Accuracy is 95.5%; Python version is 3.12.1; epoch 20."
pattern = r"\d+(?:\.\d+)+|\d+"
print(re.findall(pattern, text))
# ['95.5', '3.12.1', '20']
```

The first alternative matches a number with one or more dot-separated parts (95.5, 3.12.1). The second matches ordinary integers (20). The first alternative is listed first so the longer match is tried before the shorter one.

5.4 Extract capitalized words and vowel-starting words

```
import re
text = "Artificial Intelligence enables useful applications."
print(re.findall(r"\b[A-Z][a-zA-Z]*\b", text))
# ['Artificial', 'Intelligence']

print(re.findall(r"\b[AEIOUaeiou][A-Za-z]*\b", text))
# ['Artificial', 'Intelligence', 'enables', 'useful', 'applications']
```

5.5 Replace URLs and emails

```
import re
text = "Email help@example.com or visit https://example.com/docs"
text = re.sub(r"https?://[^\s<>]+|www\.[^\s<>]+", "<URL>", text)
text = re.sub(r"[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}", "<EMAIL>", text)
print(text)
# Email <EMAIL> or visit <URL>
```

Replace URLs first. Otherwise the email pattern could match parts of a URL that contain an @ sign.

5.6 Find all numbers with findall()

```
import re
text = "My number is 987654321 and the backup is 123456789."
print(re.findall(r"\d+", text))
# ['987654321', '123456789']
```

6. Common mistakes

Mistake	What happens	Better approach
Using `cat` without boundaries	Matches cat inside category and scatter	Use `\bcat\b` for a whole word
Confusing `*` and `+`	`*` allows zero; `+` requires at least one	Decide whether the item is optional
Using `search()` to collect every match	Only the first match is returned	Use `findall()` for all matches
Forgetting about capture groups	`findall()` returns the groups instead of the full match	Use `(?:...)` when you only need grouping
Forgetting raw strings	`\b` in a normal string means backspace, not word boundary	Write patterns as `r"..."`
Assuming regex understands meaning	Patterns match shapes of text, not semantics	Use NLP tools for context-sensitive tasks

7. Quick reference sheet

Goal	Use
First match anywhere	`re.search()`
Match at the start	`re.match()`
Validate the entire input	`re.fullmatch()`
Extract all matches	`re.findall()`
Replace matches	`re.sub()`
Split using a pattern	`re.split()`
Reuse a pattern	`re.compile()`
Escape literal user input	`re.escape()`
Digit / word char / whitespace	`\d / \w / \s`
One or more / zero or more	`+ / *`
Whole-word boundary	`\bword\b`

8. Practice questions

Try these before checking the answer key.

1. Extract all numbers from: 'Room 12 has 3 desks and 45 chairs.'
2. Extract the whole word 'cat' from: 'cat category scatter cat'.

3. Remove punctuation from: 'Hello, NLP!'
4. Replace any email address in a sentence with <EMAIL>.
5. Split 'red,blue;green yellow' on commas, semicolons, or whitespace.

Answer key

```
import re

print(re.findall(r"\d+", "Room 12 has 3 desks and 45 chairs."))
# ['12', '3', '45']

print(re.findall(r"\bcat\b", "cat category scatter cat"))
# ['cat', 'cat']

print(re.sub(r"[^\w\s]", "", "Hello, NLP!"))
# Hello NLP

print(re.sub(r"[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}",
             "<EMAIL>", "Write to help@example.com"))
# Write to <EMAIL>

print(re.split(r"[,;\s]+", "red,blue;green yellow"))
# ['red', 'blue', 'green', 'yellow']
```

Study strategy: Learn character classes and quantifiers first. Then practice `findall()`, `sub()`, and word boundaries, because these appear most often in NLP text-cleaning tasks.